



长安大学

数字逻辑实验课 实验报告

课程名称： 数字逻辑实验课
实验名称： 交通信号灯设计
专业名称： 计算机科学与技术/卓越工程师
小组成员： xxx
指导教师： 马峻岩

一、实验目的

1. 学习并掌握运用 Verilog 语言进行交通灯控制信号设计;
2. 掌握分频电路的设计方法;
3. 掌握 Moore 型有限状态机的 Verilog 设计方法;

主要仪器和设备: 计算机, Basys3 开发板。

二、实验要求

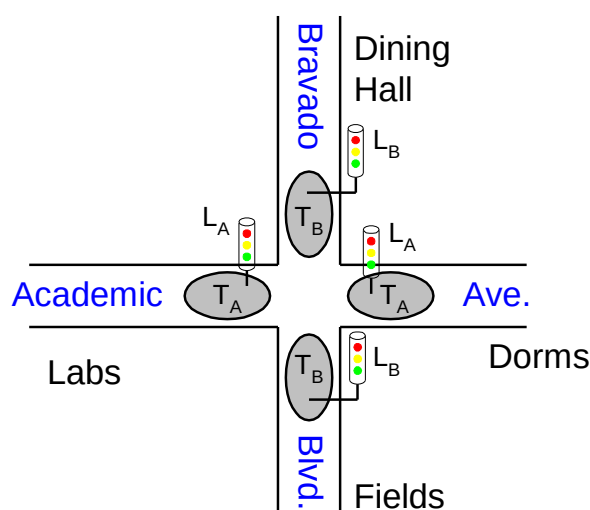


图 2-1 智能交通信号灯示意图

图 2-1 为智能交通信号灯布设示意图, 电路设计规格描述如下:

1. 输入信号: 交通灯控制系统有 2 个来自交通传感器的输入信号 T_A , T_B (为 1 时表示, 道路上有人流), 其中 T_A 为 sw0, T_B 为 sw1。
2. 交通灯控制系统有 2 组控制交通灯的输出信号 L_A , L_B (显示红、黄和绿灯), 其中 L_A 的红、黄、绿对应 led0, led1 和 led2, L_B 的红、黄、绿对应 led3, led4 和 led5。
3. 电路复位信号 reset 用于将系统重置为已知的初始状态, Academic 为绿灯, Bravado 为红灯, 其中 sw3 对应 reset 高电平有效的异步复位信号。
4. 如果 Academic 为绿灯, 且有交通, 则保持绿灯; 如果 Academic 为绿灯, 没有交通, 则 Academic 灯变黄并保持 5 秒, 然后变红。
5. 如果 Bravado 为绿灯, 且有交通, 则保持绿灯; 如果 Bravado 为绿灯, 没有交通, 则 Bravado 灯变黄并保持 5 秒, 然后变红。
6. 控制电路模块的时钟周期信号为 5 秒。

三、相关外设接口及原理

1. Basys3 开发板拨码开关原理图及简介

拨码开关的电路如图 3-1 所示。当开关打到下档时，FPGA 对应引脚输入为低电平；当开关打到上档时，FPGA 对应引脚输入为高电平。

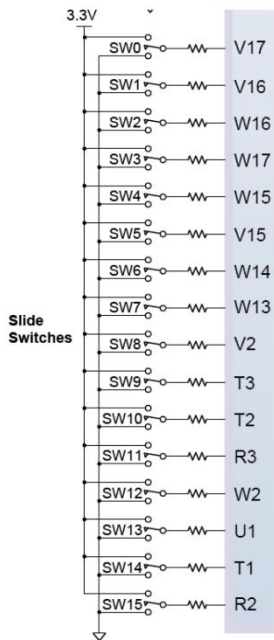


图 3-1 板拨码开关原理图

2. Basys3 开发板 LED 原理图及简介

LED 部分的电路如图 3-2 所示。当 FPGA 对应引脚输出为高电平时，相应的 LED 点亮；当 FPGA 对应引脚输出为低电平时，相应 LED 熄灭。

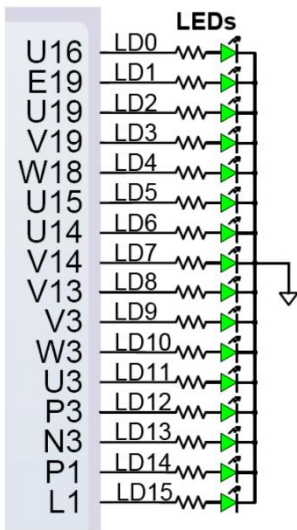


图 3-2 LED 原理图

3. 时序逻辑电路 Basys3 开发板相关约束设置

● 外部时钟信号约束文件

Basys3 的外部时钟信号为 100MHz，通过 W5 引脚连接至 FPGA，使用时需要将约束文件 Clock Signal 段的时钟约束注释删除，示例代码如下。然后在顶层模块将 clk 信号作为 input 接入。

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
```

● 外部输入信号作为边沿敏感信号的约束文件设置

如果在 always 边沿触发的敏感列表中有 btn, sw 等外部输入，如 always @(posedge clk, posedge sw[0])，需要在约束文增加一行如下代码。注意，该约束应放在约束文件的最后，同时注意约束文件格式，如 implementation 时遇到错误，把每行约束全部顶格写，不要有空格。

```
set_property CLOCK_DEDICATED_ROUTE FALSE [get_nets { sw [0] }];
```

四、实验步骤及关键代码

1. 系统模块划分及框图

根据设计规格，本实验包含四个模块，具体模块名，信号说明如表 4-1 所示。

表 4-1 系统主要模块用说明

模块名	模块主要功能说明
traffic_light_top.v	设计顶层模块
tl_controller.v	交通灯控制模块
div.v	时钟分频模块
tl_testbench.v	仿真模块

表 4-2 模块输入、输出信号说明

信号接口名称	信号所属模块	类型 (input/output)	信号说明
clk	traffic_light_top.v	input	FPGA 板载时钟信号，100MHz
sw[0]	traffic_light_top.v	input	交通传感器的输入信号 TA，为 1 时表示道路 A 上有行人
sw[1]	traffic_light_top.v	input	交通传感器的输入信号 TB，为 1 时表示道路 B 上有行人

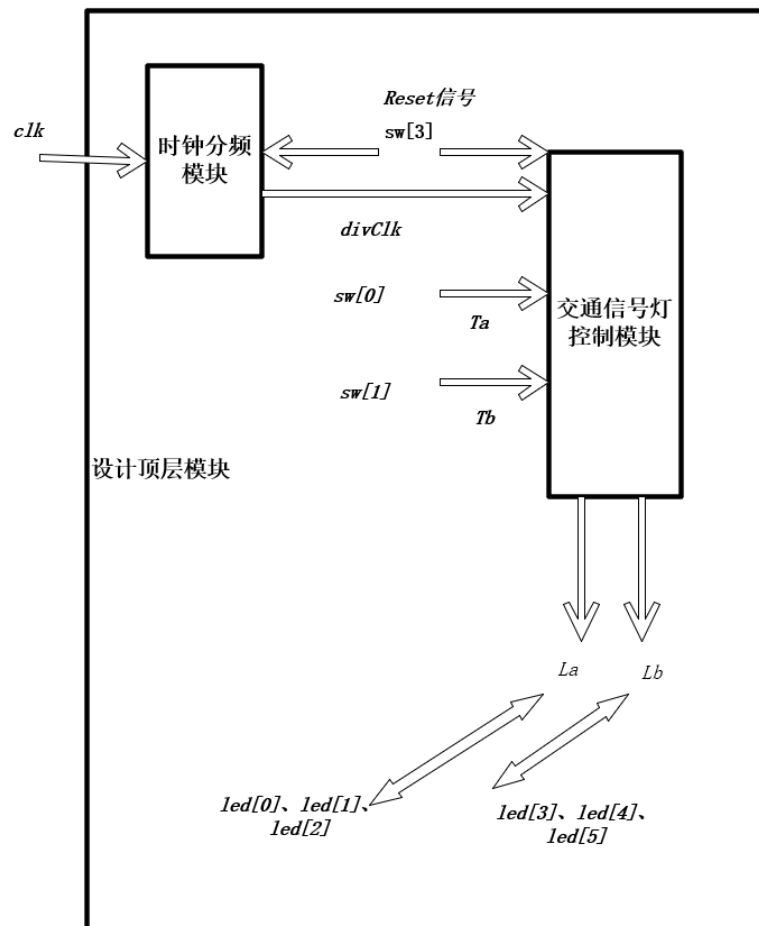


图 4-1 系统模块及连接图（含信号名）

2. 控制器设计

(1) 系统状态图设计

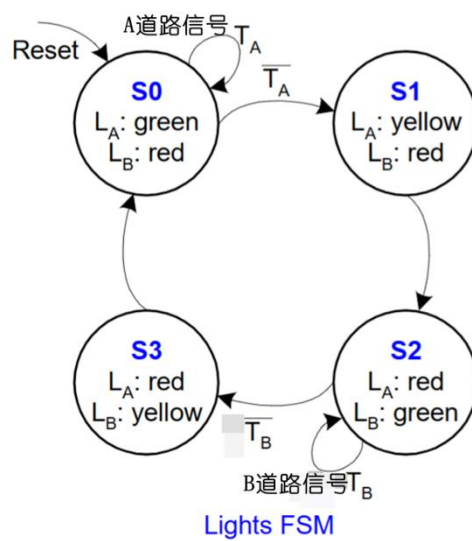


图 4-2 系统状态图

(2) 状态表

表 4-3 状态表

	Ta、 Tb 00	Ta、 Tb 01	Ta、 Tb 10	Ta、 Tb 11	La、 Lb
S0	S1	S1	S0	S0	Green、 red
S1	S2	S2	S2	S2	Yellow、 red
S2	S3	S2	S3	S2	Red、 green
S3	S0	S0	S0	S0	Red、 yellow

(3) 状态变量与状态分配

表 4-4 状态变量与状态分配表

现态 y1、 y0	次态 Y1、 Y0				Output La、 Lb
	Ta、 Tb 00	Ta、 Tb 01	Ta、 Tb 10	Ta、 Tb 11	
00	01	01	00	00	00、 10
01	10	10	10	10	01、 10
10	11	10	11	10	10、 00
11	00	00	00	00	10、 01

表 4-5 状态变量编码

状态	编码
S0	00
S1	01
S2	10
S3	11

表 4-6 状态输出颜色编码

状态输出	编码
green	00
yellow	01
red	10

表 4-7 led 输出编码

颜色编码	Led 输出编码
00	100
01	010
10	001

(4) Verilog 代码实现

```

1 module tl_controller(clk, reset, Ta, Tb, La, Lb, LedA, LedB);
2     input clk, reset;
3     input Ta, Tb;
4     output [1:0] La, Lb;
5     output reg [2:0] LedA, LedB;
6
7     reg [1:0] Y, y;
8     parameter [1:0] S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11;
9     parameter [1:0] green = 2'b00, yellow = 2'b01, red = 2'b10;
10
11     always @(y, Ta, Tb)
12     begin
13         case(y)
14             S0: if (Ta) Y <= S0;
15                 else Y <= S1;
16             S1: Y <= S2;
17             S2: if (Tb) Y <= S2;
18                 else Y <= S3;
19             S3: Y <= S0;
20         endcase
21     end
22
23     always @(posedge clk, posedge reset)
24     begin
25         if (reset)
26         begin
27             y[0] <= green;
28             y[1] <= red;
29         end
30         else
31         begin
32             y[0] <= Y[0];
33             y[1] <= Y[1];
34         end
35     end
36
37     assign La[1] = y[1];
38     assign La[0] = ~y[1] & y[0];
39     assign Lb[1] = ~y[1];
40     assign Lb[0] = y[1] & y[0];
41
42     always @(La, Lb)
43     begin
44         if (La == green)
45             LedA = 3'b100;
46         else if (La == yellow)
47             LedA = 3'b010;
48         else if (La == red)
49             LedA = 3'b001;
50
51         if (Lb == green)
52             LedB = 3'b100;
53         else if (Lb == yellow)
54             LedB = 3'b010;
55         else if (Lb == red)
56             LedB = 3'b001;
57     end
58
59 endmodule

```

```

1 module traffic_light_top(clk, sw, led[5:0]);
2     input clk;
3     input [3:0] sw;
4     output [5:0] led;
5
6     wire divClk;
7     div_clock(clk, .reset(sw[3]), .divClk(divClk));
8
9     wire [1:0] La, Lb;
10    tl_controller tl_ctrol(.clk(divClk), .reset(sw[3]), .Ta(sw[0]), .Tb(sw[1]), .La(La), .Lb(Lb), .LedA(led[2:0]), .LedB(led[3:0]));
11
12 endmodule

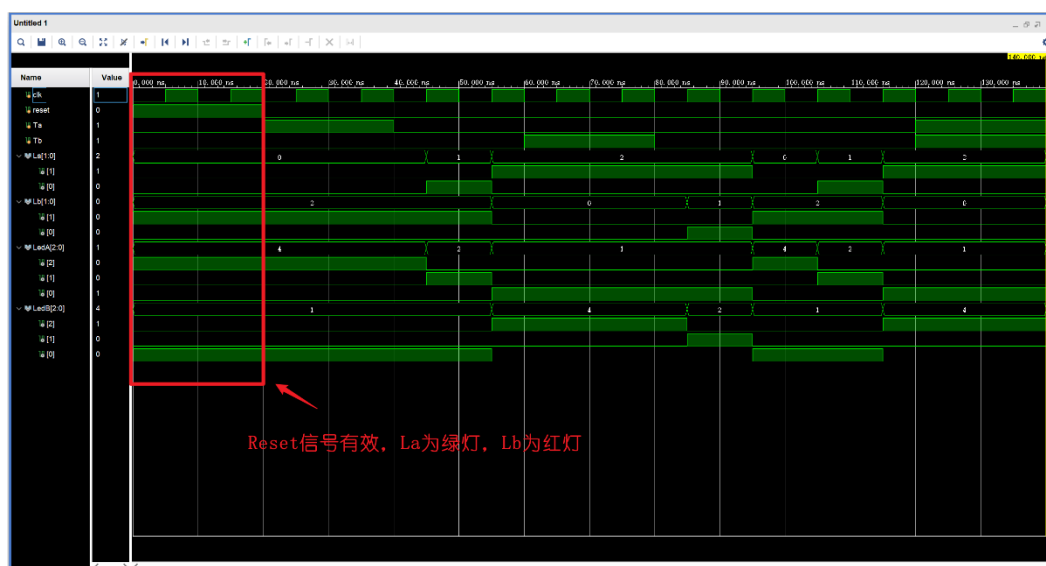
```



```
1 module tl_testbench();
2   reg clk, reset;
3   reg Ta, Tb;
4   wire [1:0] La, Lb;
5   wire [2:0] LedA, LedB;
6
7   always #5 clk = ~clk;
8
9   tl_controller tl_ctrler(clk(clk), .reset(reset), .Ta(Ta), .Tb(Tb), .La(La), .Lb(Lb), .LedA(LedA), .LedB(LedB));
10
11   initial begin
12     #0 clk = 0;
13     reset = 1;
14     Ta = 0;
15     Tb = 0;
16     #20 reset = 0; //东西方向有人，南北方向无人
17     Ta = 1;
18     Tb = 0;
19     #20 Ta = 0; //东西、南北方向均无人
20     Tb = 0;
21     #20 Ta = 0; //东西方向无人，南北方向有人
22     Tb = 1;
23     #20 Ta = 0; //东西、南北方向均无人
24     Tb = 0;
25     #20 Ta = 0; //东西、南北方向均无人
26     Tb = 0;
27     #20 Ta = 1; //东西、南北方向均有人
28     Tb = 1;
29     #20 $finish;
30   end
31 endmodule
```

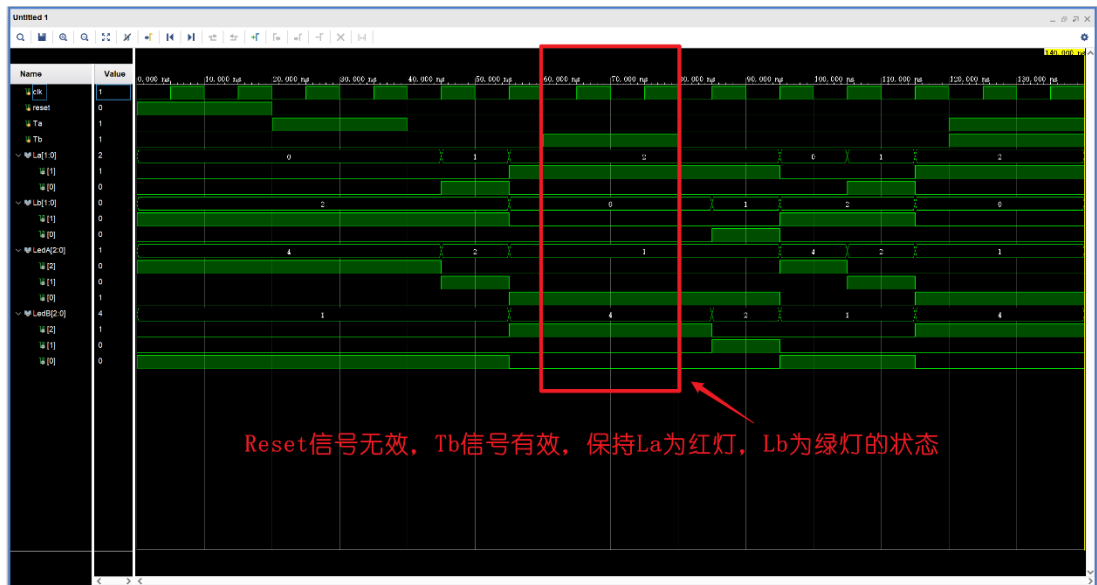
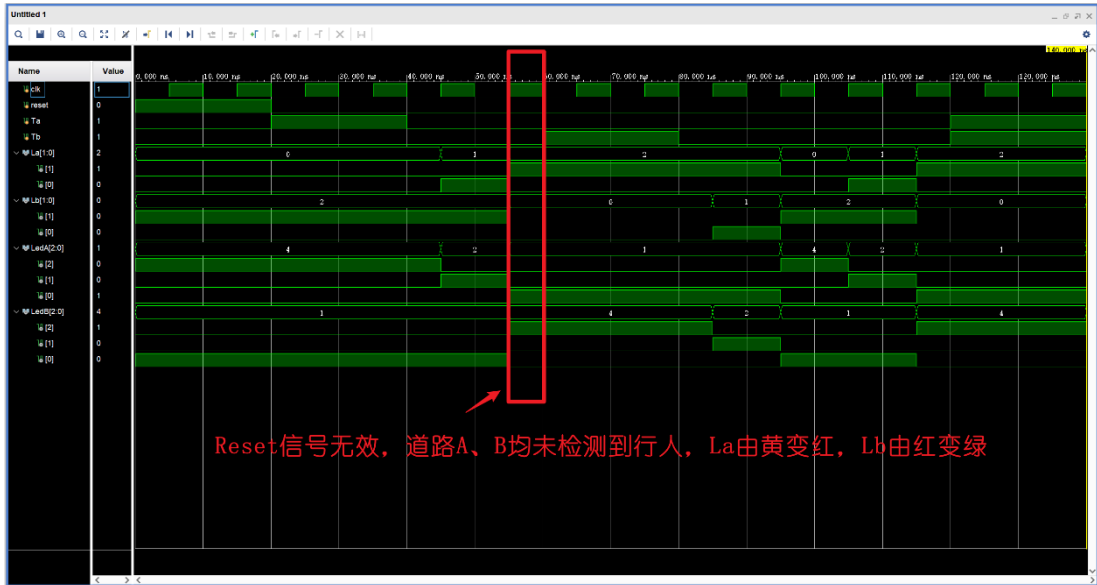
(5) 控制器测试

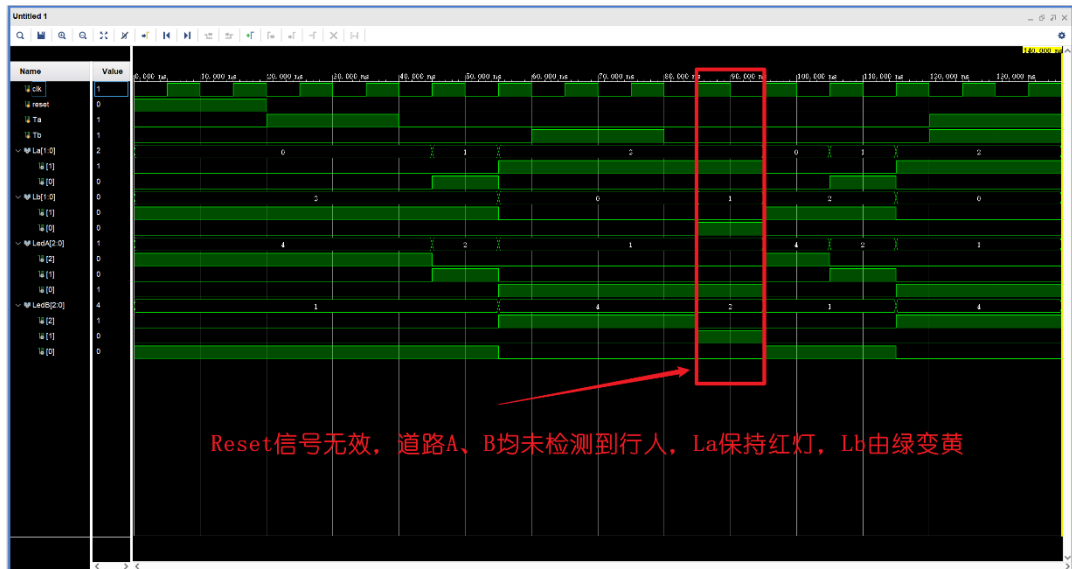
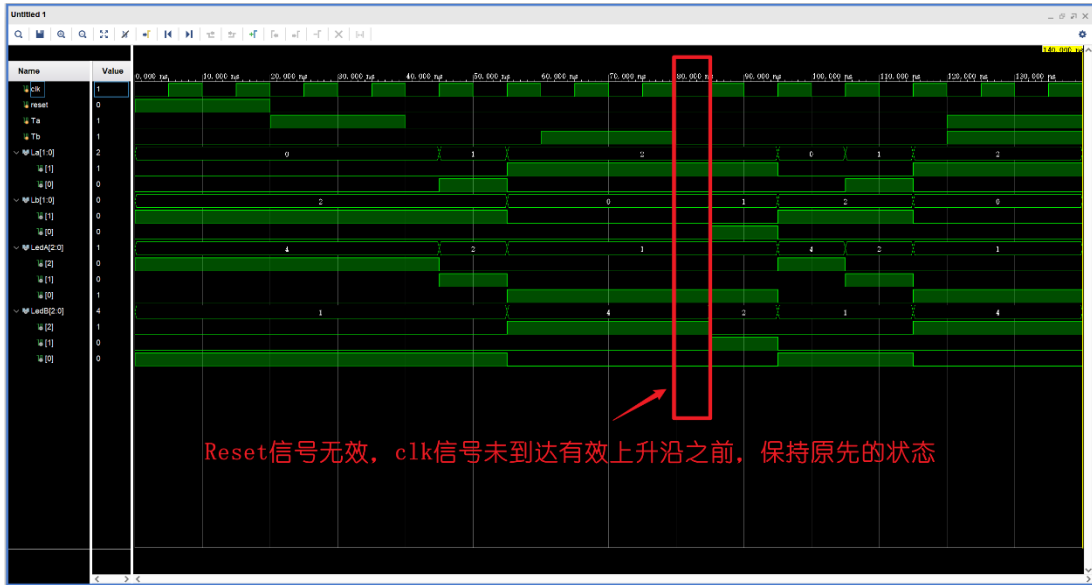
① 复位测试

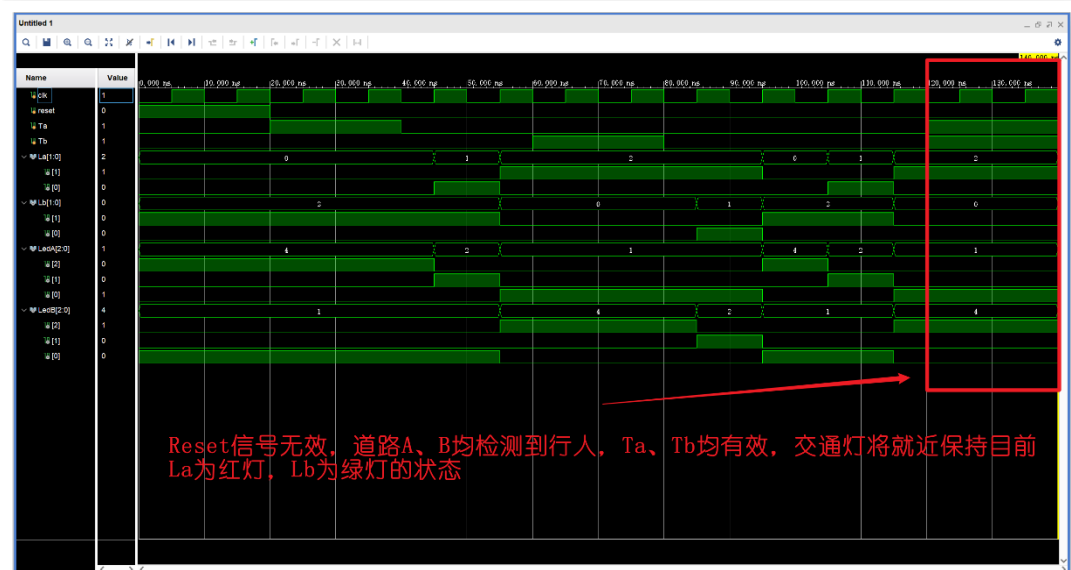
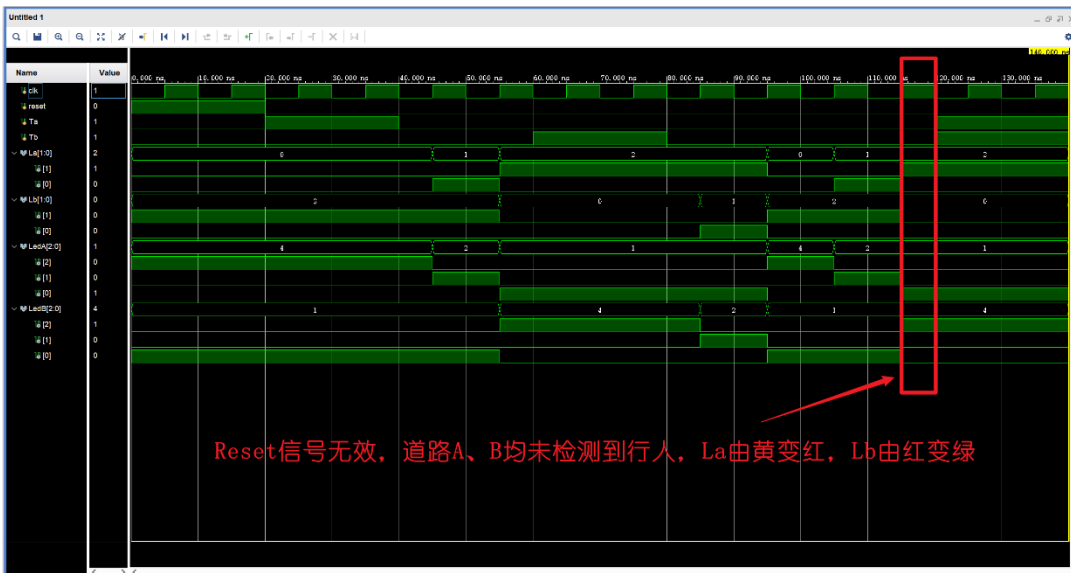


② 状态转换测试









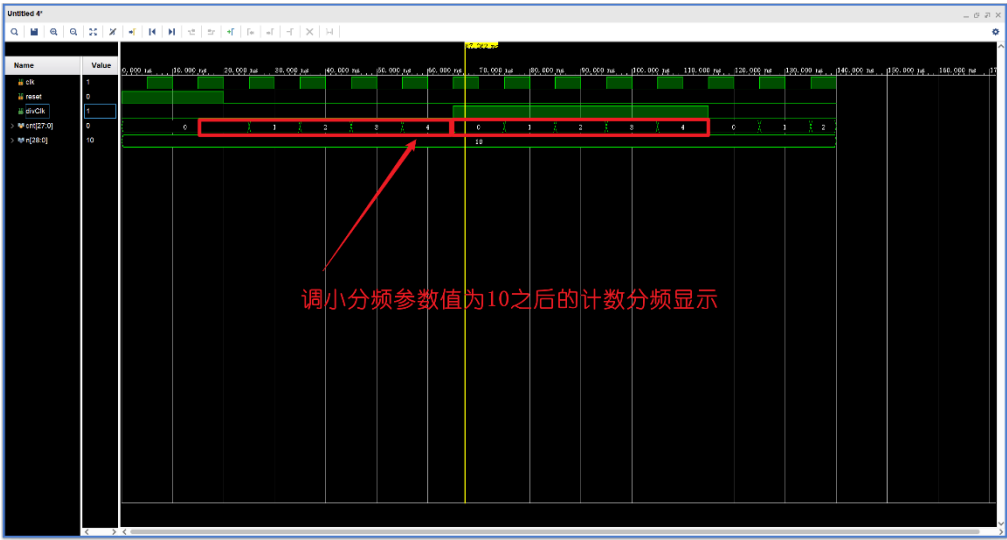
3. 分频模块设计

(1) 分频原理概述

采用计数分频

```
1 module divClk, reset, divClk);
2     input clk, reset;
3     output reg divClk;
4
5     parameter n = 29'd500_000_000;
6     reg [27:0] cnt;
7
8     always @(posedge clk, posedge reset)
9     begin
10        if (reset)
11        begin
12            divClk <= 0;
13            cnt <= 0;
14        end
15        else
16        begin
17            if (cnt == n / 2 - 1)
18            begin
19                divClk <= ~divClk;
20                cnt <= 0;
21            end
22            else
23                cnt <= cnt + 1;
24        end
25    end
26 endmodule
```

(2) 分频功能测试



4. 开发板测试与验证

表 4-3 功能测试

功能点	测试方法步骤描述	预期测试结果	测试是否通过
Reset 信号有效时的测试	sw[3]=1	La 为绿灯, Lb 为红灯 (led[2]亮, led[3]亮)	是
Reset 信号无效时的测试 (同	sw[3] = 0 sw[1] = 0	La 为绿灯, Lb 为红灯 (led[2]亮, led[3]	是

时均为检测到 Ta、Tb 的信号)	$sw[0] = 0$	亮), 经过 5 秒的时钟周期, La 为黄灯, Lb 为红灯 (led[1]亮, led[3]亮), 经过 5 秒的时钟周期, La 为红灯, Lb 为绿灯 (led[0]亮, led[5]亮), 经过 5 秒的时钟周期, La 为红灯, Lb 为黄灯 (led[0]亮, led[4]亮), 经过 5 秒的时钟周期, La 为绿灯, Lb 为红灯, 如此周期反复……	
Reset 信号无效时, Ta 信号为高电平有效, Tb 为低电平无效	$Reset = 0$ $sw[0] = 1$ $sw[1] = 0$	路灯状态经过一定的时钟周期, 当转移到 La 为绿灯, Lb 为红灯 (led[2]亮, led[3]亮) 的状态时, 一直保持该状态	是
Reset 信号无效时, Ta 信号为低电平无效, Tb 信号为高电平有效	$Reset = 0$ $sw[0] = 0$ $sw[1] = 1$	路灯状态经过一定的时钟周期, 当转移到 La 为红灯, Lb 为绿灯 (led[0]亮, led[5]亮) 的状态时, 一直保持该状态	是

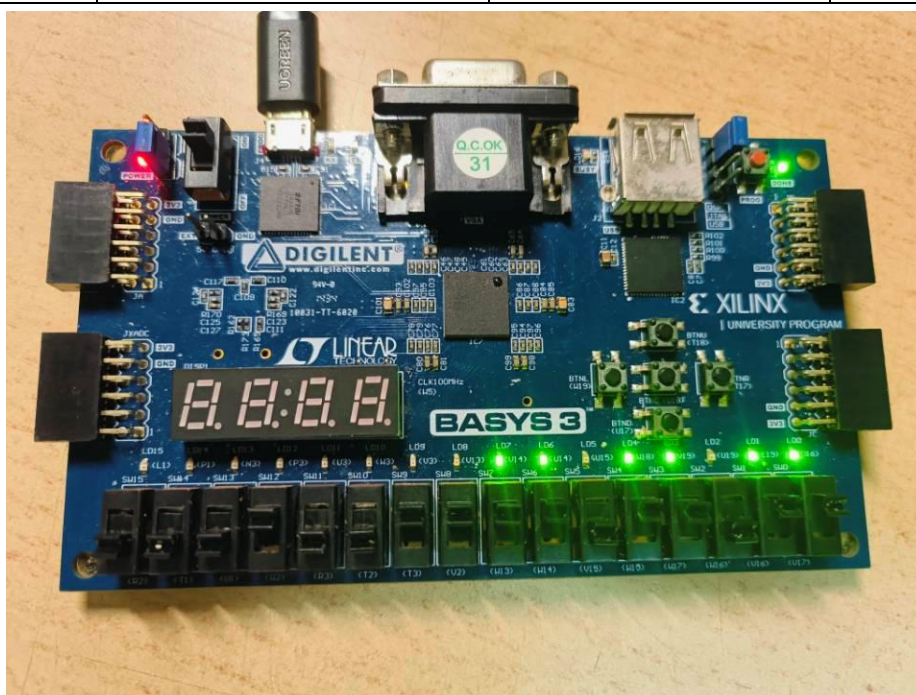


图 1 test1

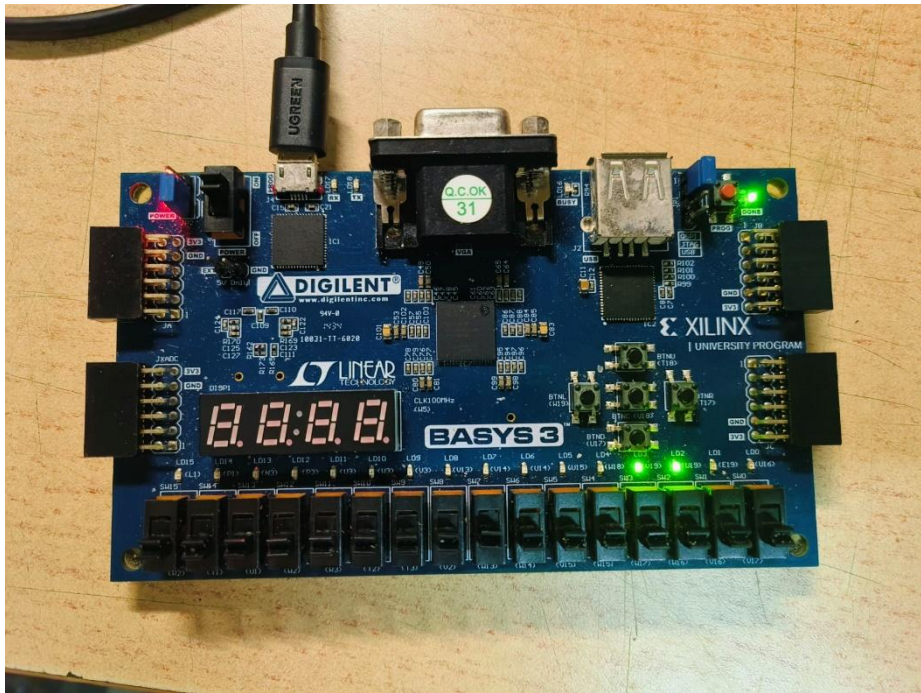


图 2 test2.1

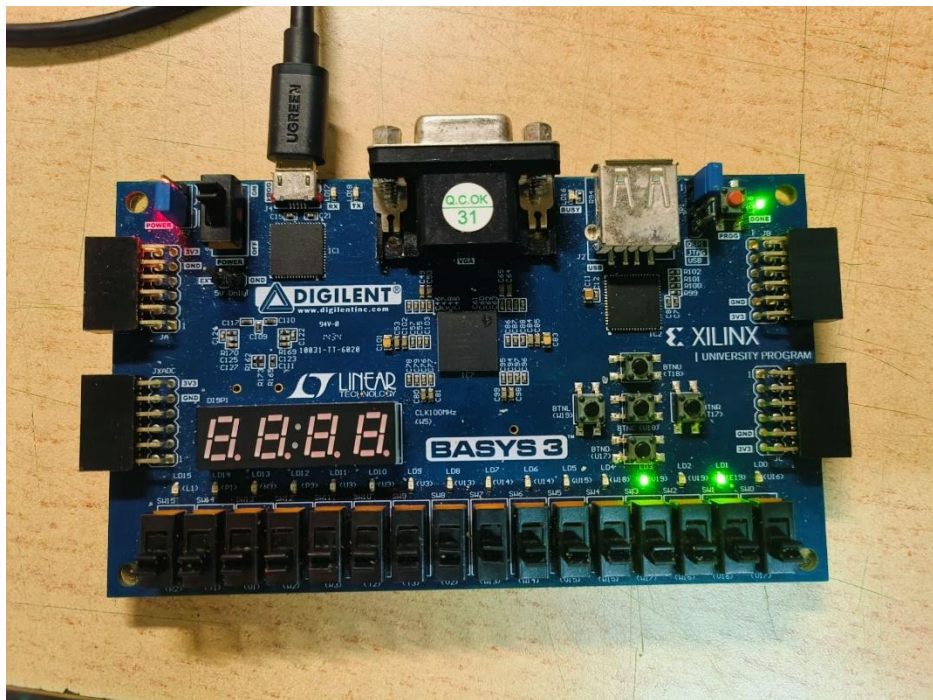


图 3 test2.2

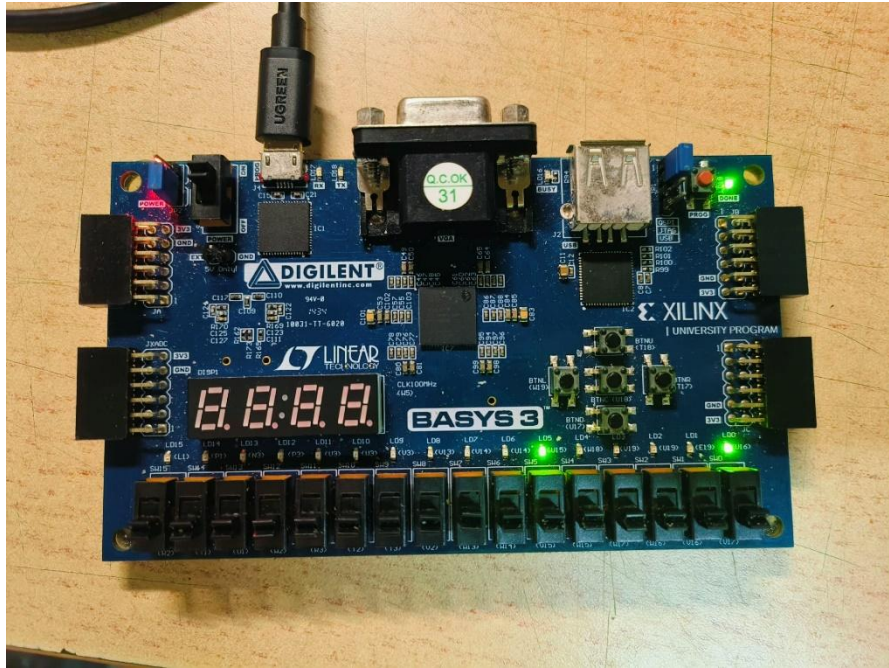


图 4 test2.3

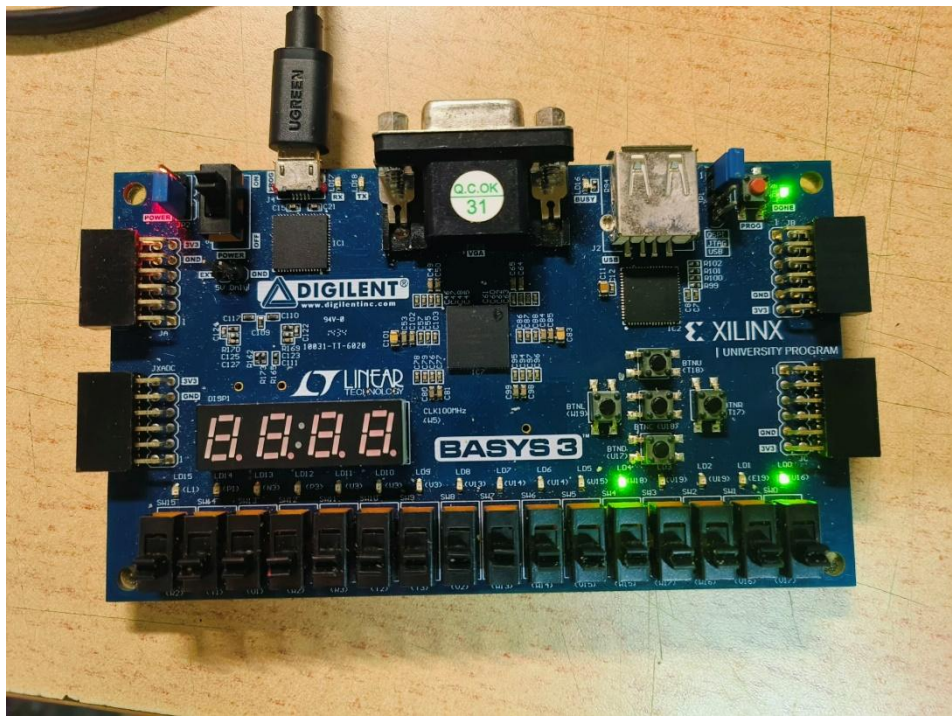


图 5 test2.4

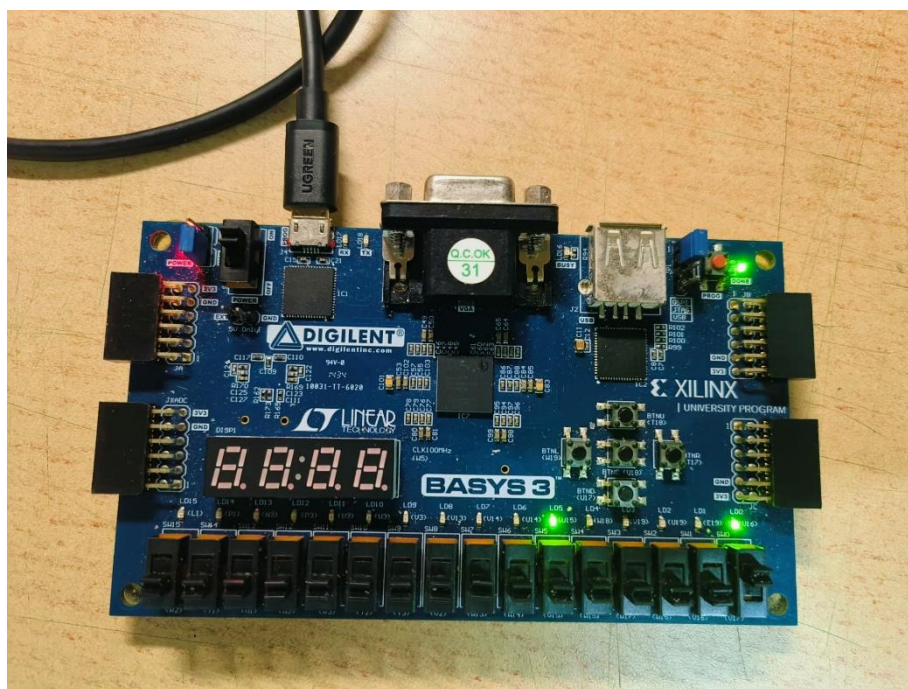


图 6 test3.1

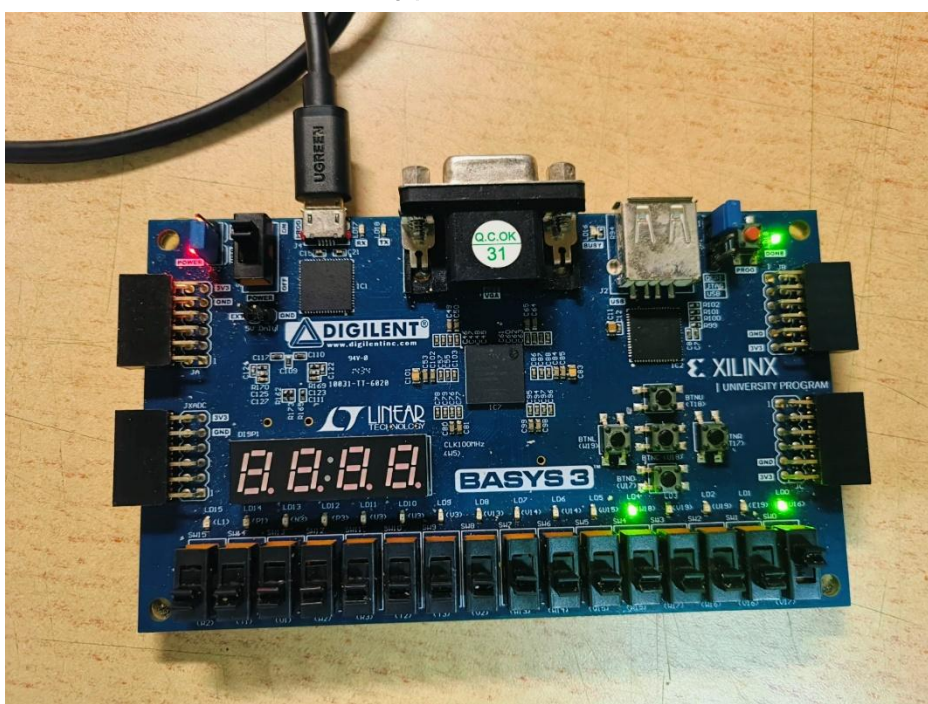


图 7 test3.2

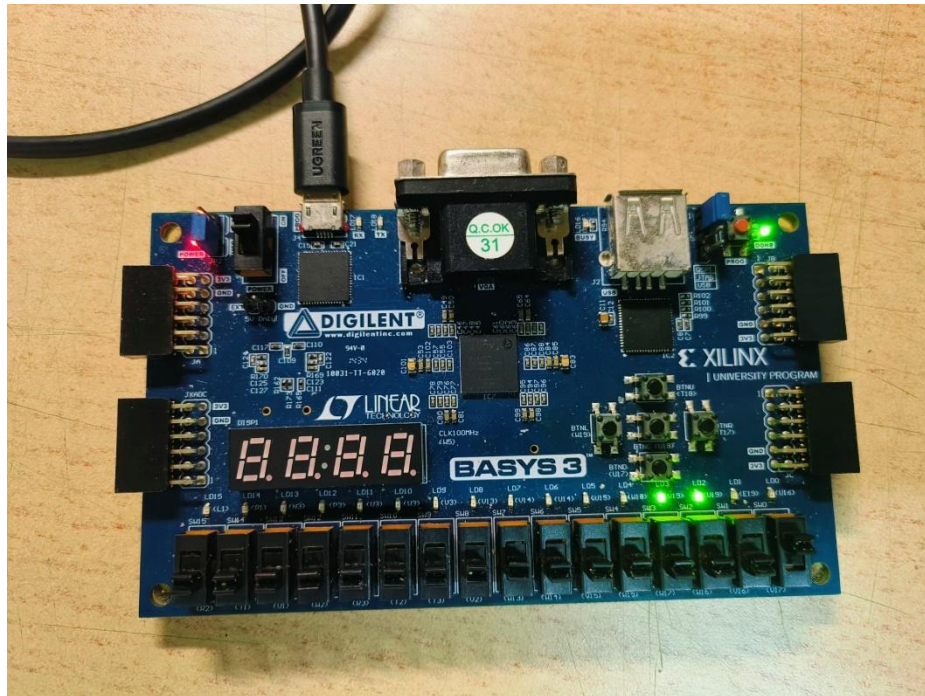


图 8 test3.3

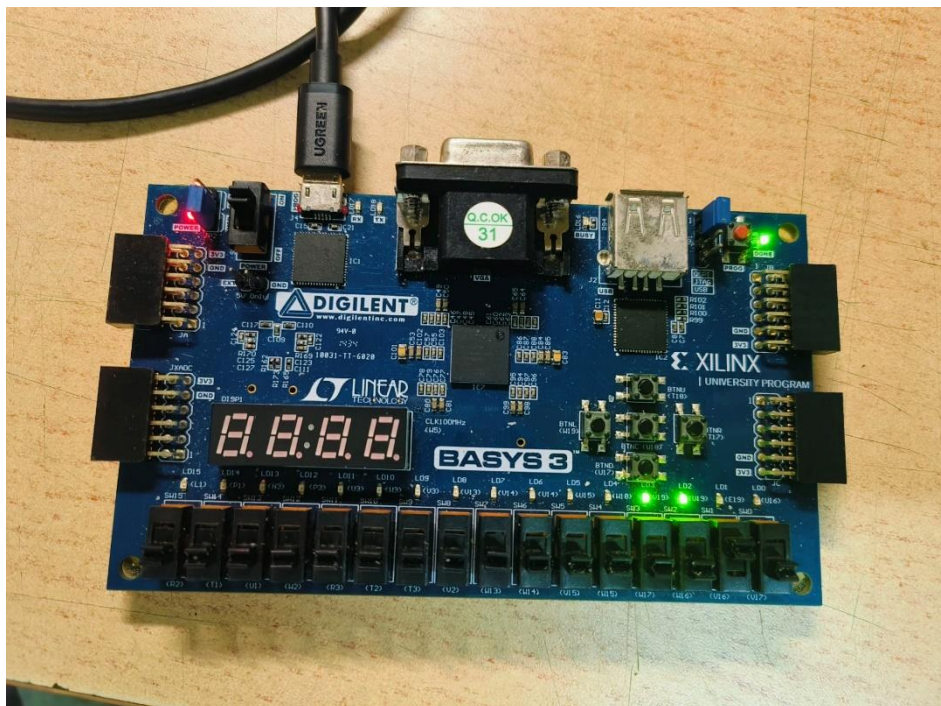


图 9 test4.1

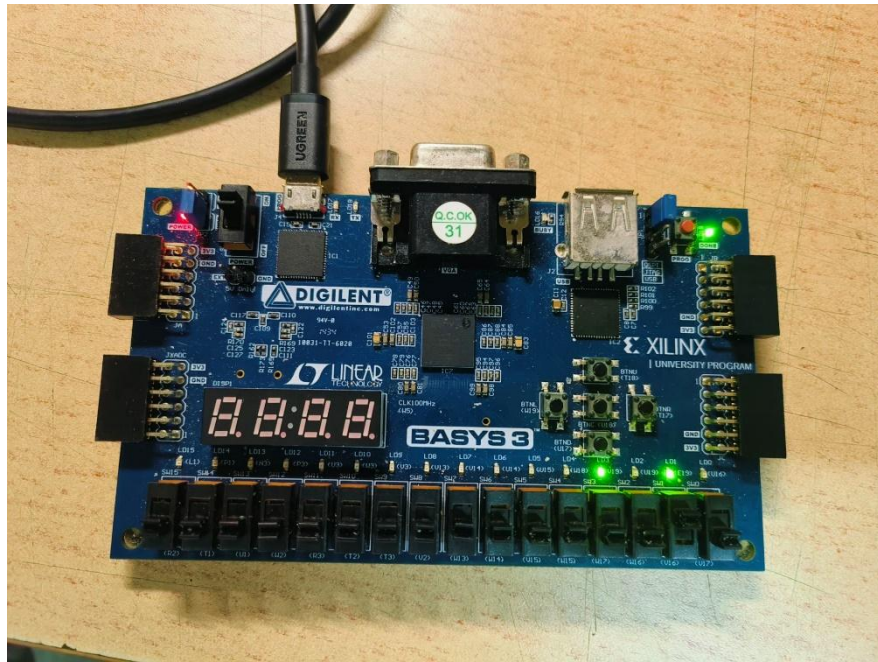


图 10 test4.2

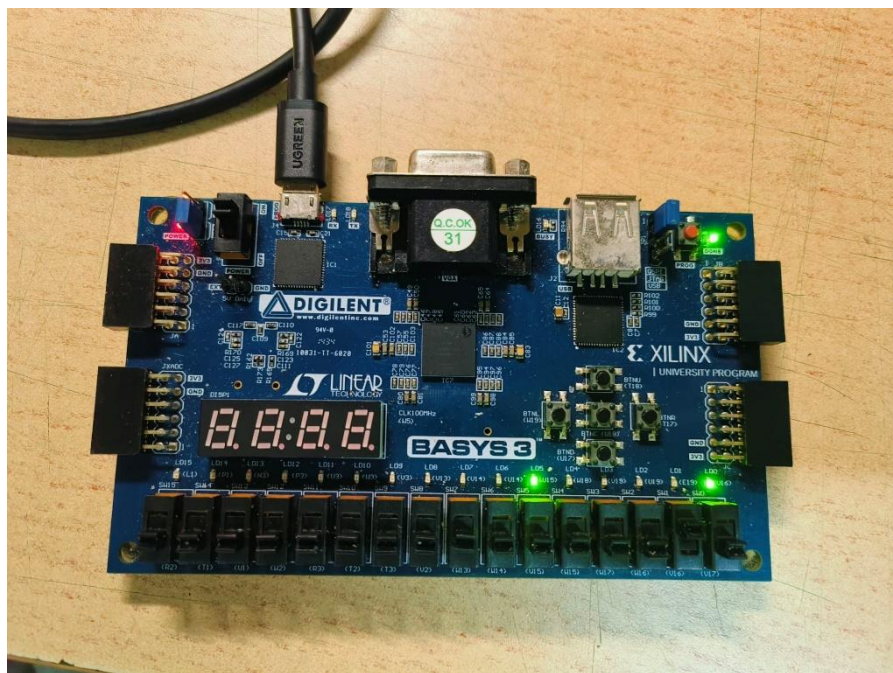


图 11 test4.3

五、遇到的问题及解决方法

1. 信号输入模块

输入信号有两个交通传感器的输入信号 TA 和 TB，分别对应于 sw0 和 sw1，还有一个复位信号 reset，对应于 sw3。

问题： 如何可靠地检测这些输入信号？

解决方法:

确保输入信号的去抖动处理, 避免由于机械开关引起的误触发。

使用同步电路来采样异步输入信号, 避免亚稳态问题。

2. 状态机设计

问题: 如何实现状态机的设计和状态转换?

解决方法:

使用 Verilog 来实现状态机。

定义状态转换条件, 根据 TA 和 TB 的输入信号决定是否保持当前状态或进行状态转换。

定义每个状态的持续时间, 特别是黄灯状态需要保持 5 秒。

3. 时钟信号

问题: 如何生成和管理时钟信号?

解决方法:

使用一个计数器来生成 5 秒的时钟信号。

在 FPGA 上, 可以使用分频器来实现较长时间的时钟周期。

4. 输出控制

问题: 如何根据状态机的状态控制输出信号?

解决方法:

在状态机的每个状态中定义对应的输出信号。

使用 VHDL 或 Verilog 中的 assign 语句或 process 块来设置输出信号。

5. 异步复位

问题: 如何实现异步复位?

解决方法:

Verilog 代码中添加复位逻辑。使用一个异步复位信号, 当复位信号有效时, 将状态机重置为初始状态